UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of Computing

Semester 01 2025/2026

Subject : Database Programming (SECP3623)
Section : Section 1-2
Task : Project II

Name	Matric No.
GOE JIE YING	A23CS0224
LAM YOKE YU	A23CS0233
TAN YI YA	A23CS0187
TEH RU QIAN	A23CS0191
WOO CHENG SHUAN	A23CS0283

Table of Contents

1.0 Introduction

1.1 Domain Description

The chosen domain is the car rental system, which manages vehicles, customers, bookings and payments. This system allows customers to rent vehicles for specific periods, track booking statuses and handle payments including deposits and refunds.

The Entity Relationship Diagram (ERD) shows the relationship of the car rental system. It includes four main entities: Vehicles, Customers, Bookings and Payments. Each vehicle can be booked by a customer and each booking may have one or more payments.

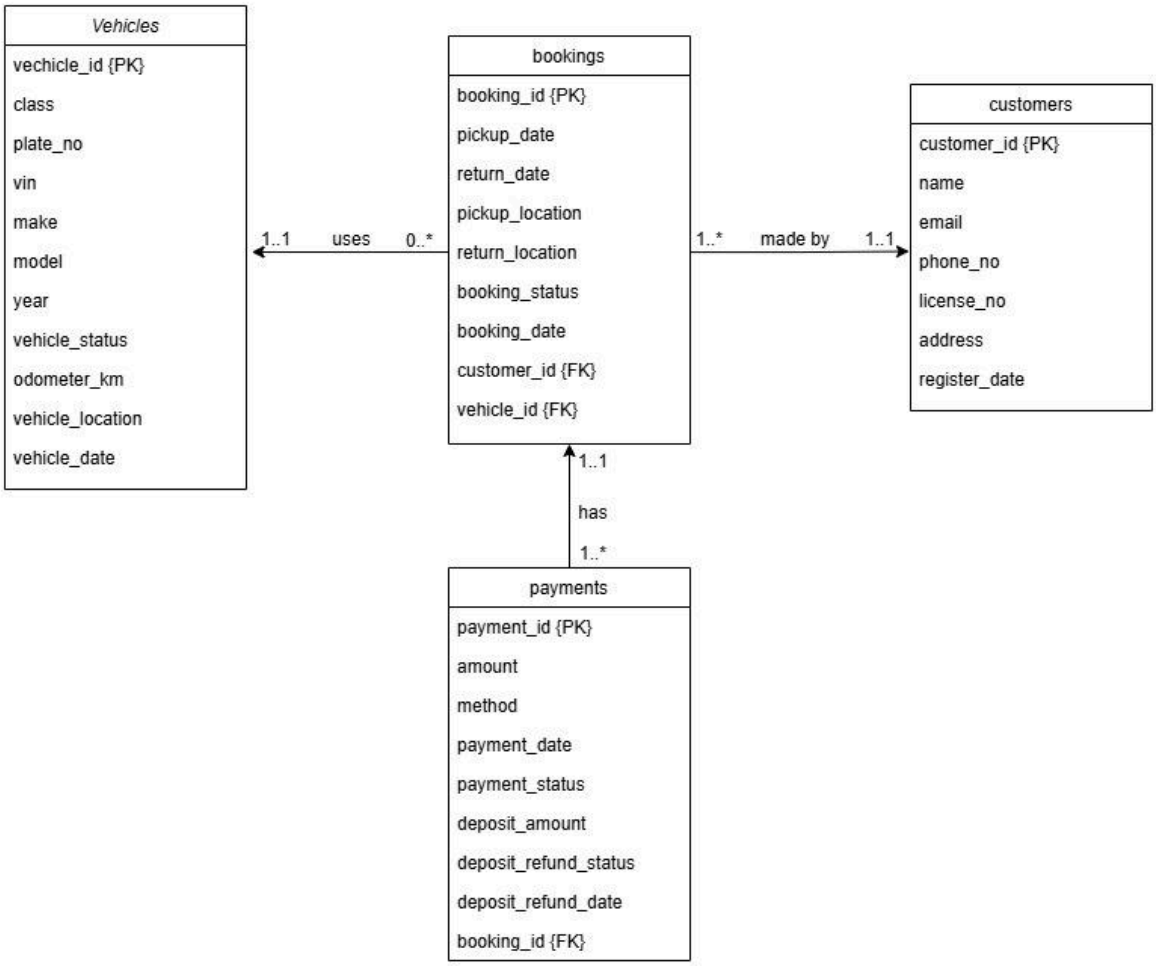


Figure 1.0 ERD of Car Rental System

Unlike traditional relational databases, MongoDB is well-suited for this domain because it can handle flexible and hierarchical data structures, such as:

- **Embedded documents** - for storing related documents together
- **References** - for linking collections while maintaining modularity
- **Arrays** - for multi-valued attributes

1.2 Project Objectives

The goals of this project are:

- To design a NoSQL backend using MongoDB for a car rental system.
- To apply document data modelling principles including embedding, referencing and arrays
- To demonstrate CRUD operations (insert, query, update, delete) in MongoDB
- To implement indexing and aggregation pipelines for performance and analytics
- To analyse security considerations and design challenges in NoSQL systems

1.3 Team members & roles

The table shows the role of team members.

Team Member	Role and Responsibilities
Teh Ru Qian	Responsible for providing the project overview, domain description, objectives and designing the NoSQL data model.
Tan Yi Ya	Responsible for implementing the basic MongoDB CRUD operations and summarizing the project outcomes in the conclusion.
Woo Cheng Shuan	Responsible for designing and demonstrating indexing strategies, analyzing query performance and optimizing retrieval in MongoDB.
Lam Yoke Yu	Responsible for creating and demonstrating aggregation pipelines to perform complex analytics and generate operational insights from the database.
Goe Jie Ying	Responsible for implementing sorting operations, discussing security measures for sensitive data and analyzing the limitations of MongoDB for the project context.

2.0 NoSQL Data Model

2.1 Collections

The Car Rental System is implemented using four MongoDB collections based on the ERD:

1. **Vehicles** - Stores vehicle details
2. **Customers** - Stores customer information including license details and contact information
3. **Bookings** - Records rental transactions, and linking customers and vehicles
4. **Payments** - Tracks payment details, deposits and refund history for bookings.

2.2 Example JSON Documents

2.2.1 Vehicles Collection

Vehicle data using ObjectId, integers and strings to store identification, specifications, status, mileage and location.

```JSON
{
  "_id": {
    "$oid": "695bb259f224b89ed96f601a"
  },
  "vehicle_id": 1,
  "class": "Sedan",
  "plate_no": "ABC123",
  "vin": "1HGCM82633A004352",
  "make": "Toyota",
  "model": "Vios",
  "year": 2020,
  "vehicle_status": "available",
  "odometer_km": 25000,
  "vehicle_location": "Kuala Lumpur"
}
```

2.2.2 Customers Collection

Customer data using ObjectId, integers, strings and an array to store identification, contact details, license number and address.

```json
{
  "_id": {
    "$oid": "695baf4ff224b89ed96f600e"
  },
  "customer_id": 1,
  "name": "Nur Aisyah Binti Ahmad",
  "email": "aisyah.ahmad@example.com",
  "phone_no": [
    "0123456789",
    "0176543210"
  ],
  "license_no": "L1234567",
  "address": "Selangor"
}
```

2.2.3 Bookings Collection

Booking data using ObjectId, integers, strings and embedded documents to store booking ID, customer and vehicle references, pickup and return details, status and booking date.

```JSON
{
  "_id": {
    "$oid": "695bb512f224b89ed96f6029"
  },
  "booking_id": 1,
  "customer_id": 1,
  "vehicle_id": 2,
  "pickup": {
    "date": {
      "$date": "2025-01-01T00:00:00.000Z"
    },
    "location": "KL TRX"
  },
  "return": {
    "date": {
      "$date": "2025-01-03T00:00:00.000Z"
    },
    "location": "KL TRX"
  },
  "booking_status": "completed",
  "booking_date": {
    "$date": "2024-12-20T10:15:00.000Z"
  }
}
```

2.2.4 Payments Collection

Payment data using ObjectId, integers, strings and embedded documents to store payment ID, booking reference, amount, method, payment date, status and deposit with refund details.

```JSON
{
  "_id": {
    "$oid": "695bbc83f224b89ed96f6035"
  },
  "payment_id": 1,
  "booking_id": 1,
  "amount": 300,
  "method": "Credit Card",
  "payment_date": {
    "$date": "2025-01-01T00:00:00.000Z"
  },
  "payment_status": "paid",
  "deposit": {
    "amount": 100,
    "refund": {
      "status": "refunded",
      "date": {
        "$date": "2025-01-03T00:00:00.000Z"
      }
    }
  }
}
```

2.3 Embedding vs Referencing

Embedding document used when data is closely related to and dependent on its parent entity. It is stored within the same record, making reads faster and keeping related data together. For example, booking pickup or return details and payment deposit or refund information are embedded because they are usually accessed together.

Referencing is used when modeling relationships between collections where data is shared and duplication should be avoided. For example, a booking references a customer and a vehicle so that customer and vehicle details are maintained in one place, while payments reference bookings to link transactions without repeating booking data.

Aspect	Embedding	Referencing
Relationship type	Tightly coupled, dependent data	Loosely coupled, reusable data

Read performance	Faster (single document query)	Slower (needs $lookup or multiple queries)
Data duplication	Acceptable and common	Avoided
Data ownership	Child data exists only with parent	Data exists independently
Best use case	One-to-few data accessed together	One-to-many or shared data

2.4 Data Types Used

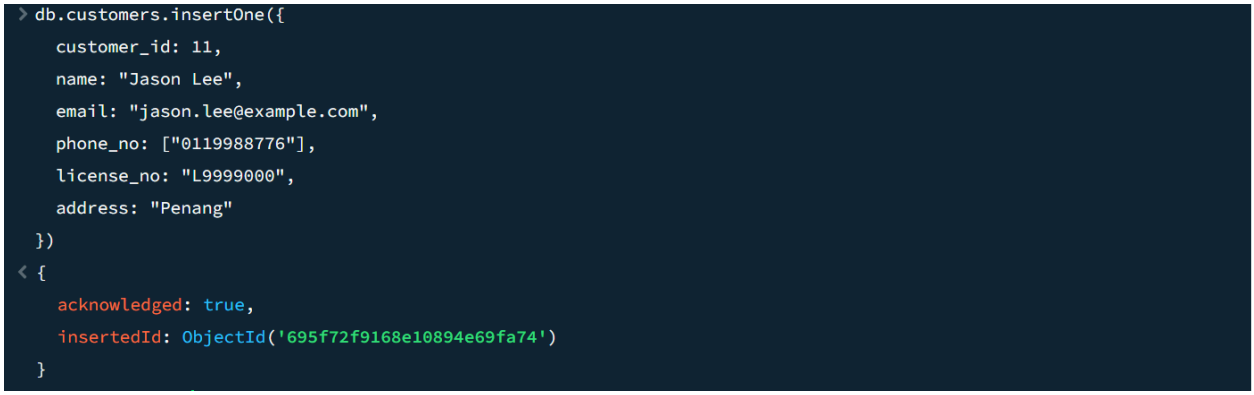
Data Type	Fields	Description
ObjectId	_id	Automatically generated unique identifier for each MongoDB document.
String	booking_status, location, name, email, license_no, address, method, vehicle_status, make, model, vin, plate_no	Used to store textual data such as names, statuses, locations, vehicle details and payment methods.
Integer	booking_id, customer_id, vehicle_id, payment_id, amount, deposit.amount, odometer_km, year	Used for numeric identifiers, monetary values, vehicle mileage and manufacturing year.
Date	pickup.date, return.date, booking_date, payment_date, deposit.refund.date	Used to represent booking periods. Payment dates and refund dates in a standard date-time format.
Array	phone_no	Used to store multiple phone numbers for a single customer.
Object (Embedded Document)	pickup, return, deposit, deposit.refund	Used to group related data together, such as pickup/return details and deposit refund information.

3.0 CRUD Implementation Summary

3.1 Insert Operations

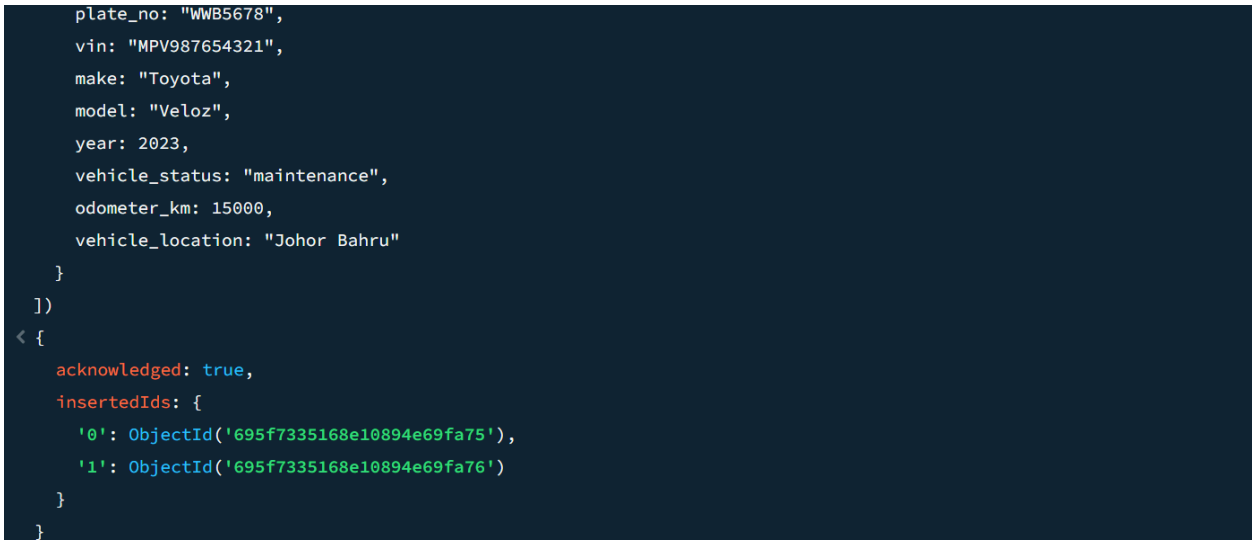
3.1.1 Inserting one customer data using **insertOne()**

```JSON
db.customers.insertOne({
  customer_id: 11,
  name: "Jason Lee",
  email: "jason.lee@example.com",
  phone_no: ["0119988776"],
  license_no: "L9999000",
  address: "Penang"
})
```



3.1.2 Inserting multiple vehicle data at once using **insertMany()**

```JSON
db.vehicles.insertMany([
  {
    vehicle_id: 11,
    class: "MPV",
    plate_no: "WWA1234",
    vin: "MPV123456789",
    make: "Perodua",
    model: "Alza",
    year: 2024,
    vehicle_status: "available",
    odometer_km: 500,
    vehicle_location: "Kuala Lumpur"
  },
  {
    vehicle_id: 12,
    class: "MPV",
    plate_no: "WWB5678",
    vin: "MPV987654321",
    make: "Toyota",
    model: "Veloz",
    year: 2023,
    vehicle_status: "maintenance",
    odometer_km: 15000,
    vehicle_location: "Johor Bahru"
  }
])
```

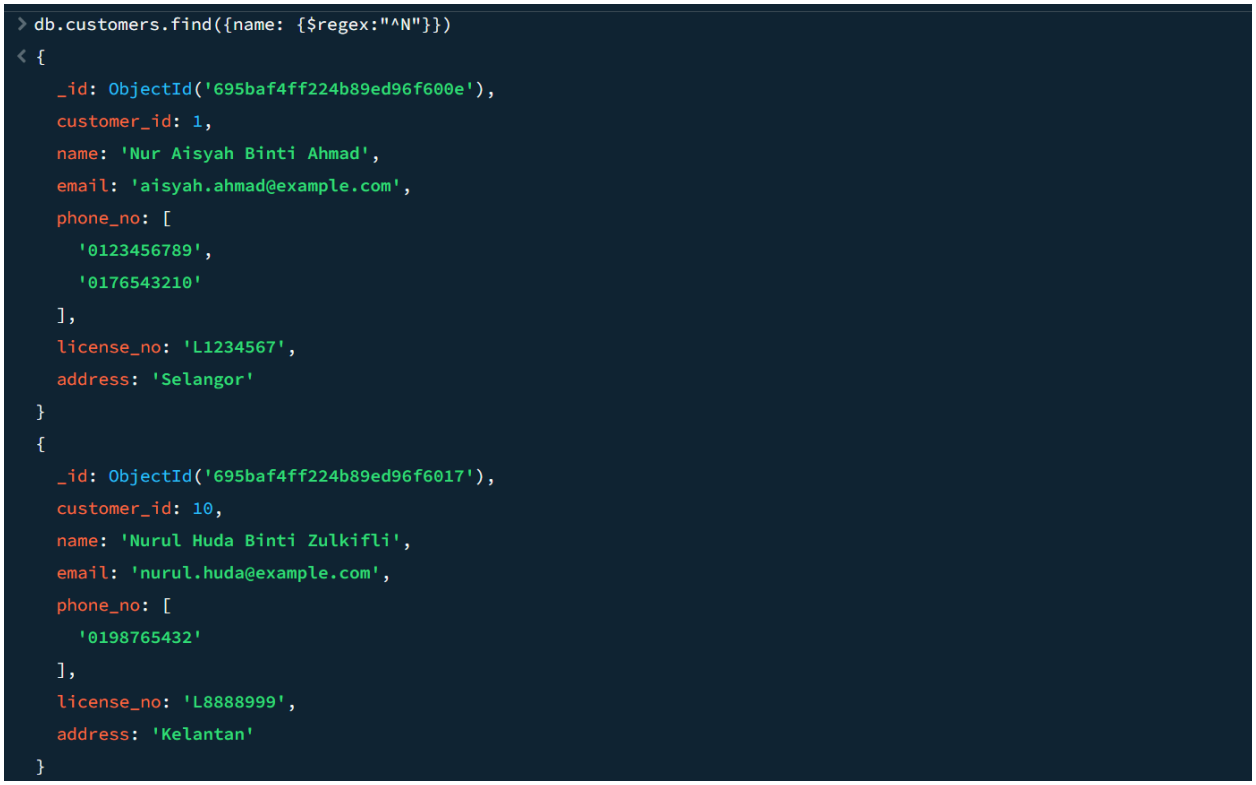


3.2 Query Operations

3.2.1 Finding customers whose name starts with the letter “N” with find()

```
JSON
db.customers.find({name: {$regex:"^N"}})
```

$regex refers to using regular expressions for pattern matching in text, while regex element “^” means performing a match at the start of a string, in this case, finding the first letter of name that matches alphabet N. find() indicates finding all names starting with N in the customer collection.



3.2.2 Finding customers whose name starts with the letter “N” with findOne()

```
JSON
db.customers.findOne({name: {$regex:"^N"}})
```

This query is almost the same as the query 2.1, but findOne() means it will only find and display the first name starting with N in the customer collection instead of all names.

```
> db.customers.findOne({name: {$regex:"^N"}})
< {
    _id: ObjectId('695baf4ff224b89ed96f600e'),
    customer_id: 1,
    name: 'Nur Aisyah Binti Ahmad',
    email: 'aisyah.ahmad@example.com',
    phone_no: [
      '0123456789',
      '0176543210'
    ],
    license_no: 'L1234567',
    address: 'Selangor'
  }
```

3.2.3 Conditional Filtering

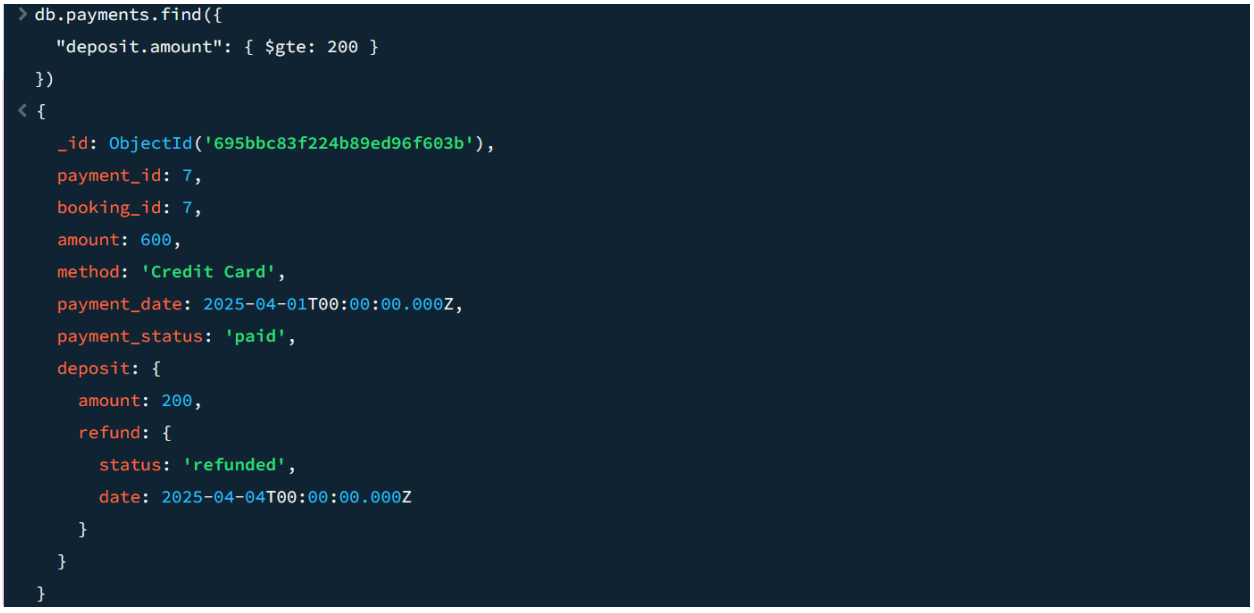
3.2.3.1 Filtering vehicles that has mileage greater than 50000km with $gt

```
JSON
db.vehicles.find({odometer_km: {$gt: 50000}})
```

```
> db.vehicles.find({ odometer_km: { $gt: 50000 } })
< {
    _id: ObjectId('695bb259f224b89ed96f601e'),
    vehicle_id: 5,
    class: 'Van',
    plate_no: 'POI222',
    vin: '4T1BF1FK5EU365555',
    make: 'Toyota',
    model: 'Hiace',
    year: 2018,
    vehicle_status: 'available',
    odometer_km: 66000,
    vehicle_location: 'Melaka'
  }
  {
    _id: ObjectId('695bb259f224b89ed96f6020'),
    vehicle_id: 7,
    class: 'Sedan',
    plate_no: 'QWE444',
    vin: '3VWFE21C04M000999',
    make: 'BMW',
    model: '320i',
    year: 2017,
    vehicle_status: 'rented',
    odometer_km: 55000,
    vehicle_location: 'Kuala Lumpur'
  }
```

3.2.3.2 Filtering payment records where deposit amount is more than or equal to 200 using $gte

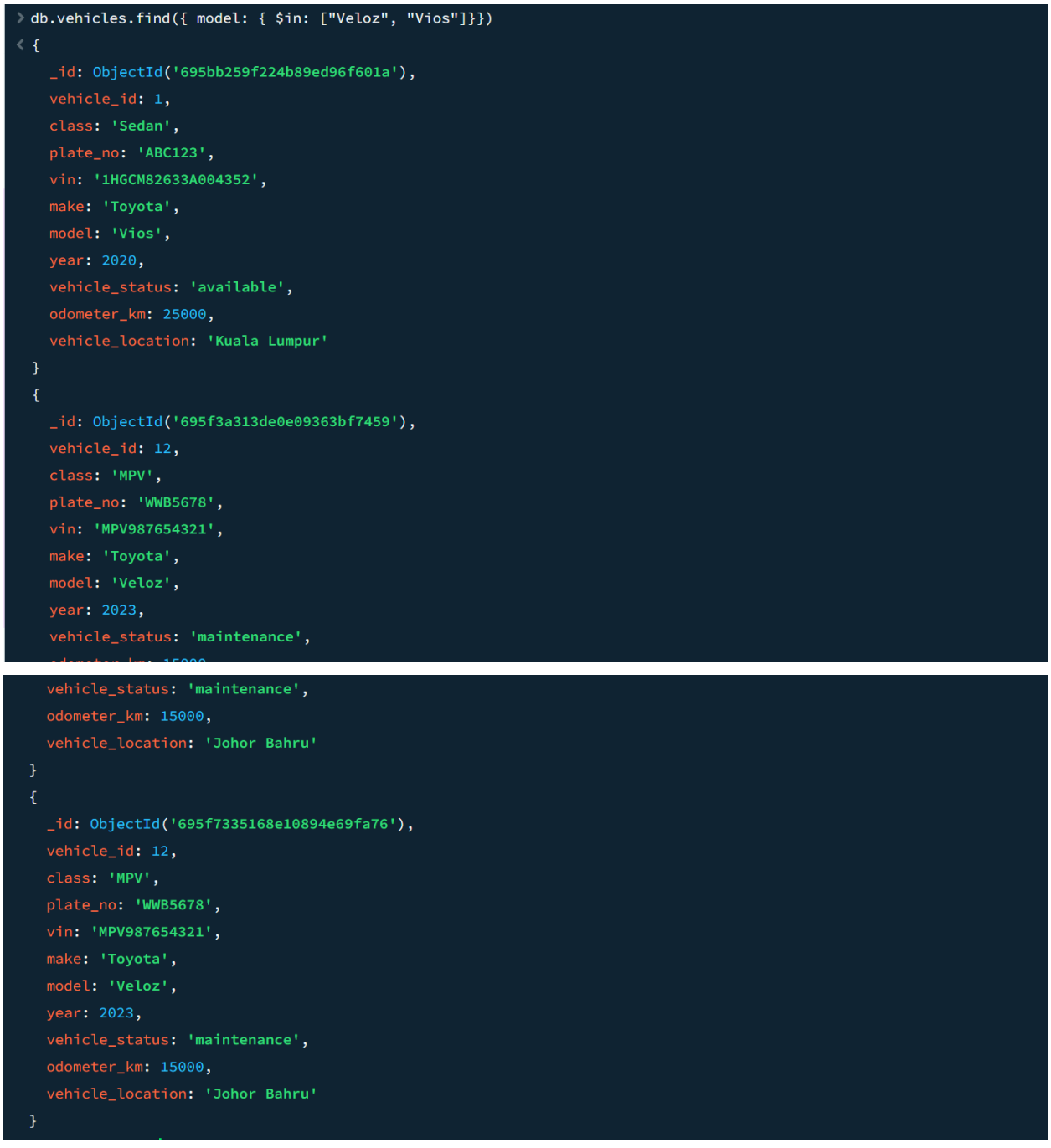
```JSON
db.payments.find({
  "deposit.amount": {$gte: 200}
})
```



```
> db.payments.find({
    "deposit.amount": { $gte: 200 }
  })
< {
    _id: ObjectId('695bbc83f224b89ed96f603b'),
    payment_id: 7,
    booking_id: 7,
    amount: 600,
    method: 'Credit Card',
    payment_date: 2025-04-01T00:00:00.000Z,
    payment_status: 'paid',
    deposit: {
      amount: 200,
      refund: {
        status: 'refunded',
        date: 2025-04-04T00:00:00.000Z
      }
    }
  }
```

3.2.3.3 Filtering vehicles that the model is either Veloz or Vios using $in

```
JSON
db.vehicles.find({model: {$in: ["Veloz", "Vios"]}})
```



```
> db.vehicles.find({ model: { $in: ["Veloz", "Vios"]}})
< {
    _id: ObjectId('695bb259f224b89ed96f601a'),
    vehicle_id: 1,
    class: 'Sedan',
    plate_no: 'ABC123',
    vin: '1HGCM82633A004352',
    make: 'Toyota',
    model: 'Vios',
    year: 2020,
    vehicle_status: 'available',
    odometer_km: 25000,
    vehicle_location: 'Kuala Lumpur'
  }
  {
    _id: ObjectId('695f3a313de0e09363bf7459'),
    vehicle_id: 12,
    class: 'MPV',
    plate_no: 'WWB5678',
    vin: 'MPV987654321',
    make: 'Toyota',
    model: 'Veloz',
    year: 2023,
    vehicle_status: 'maintenance',
    odometer_km: 15000
```

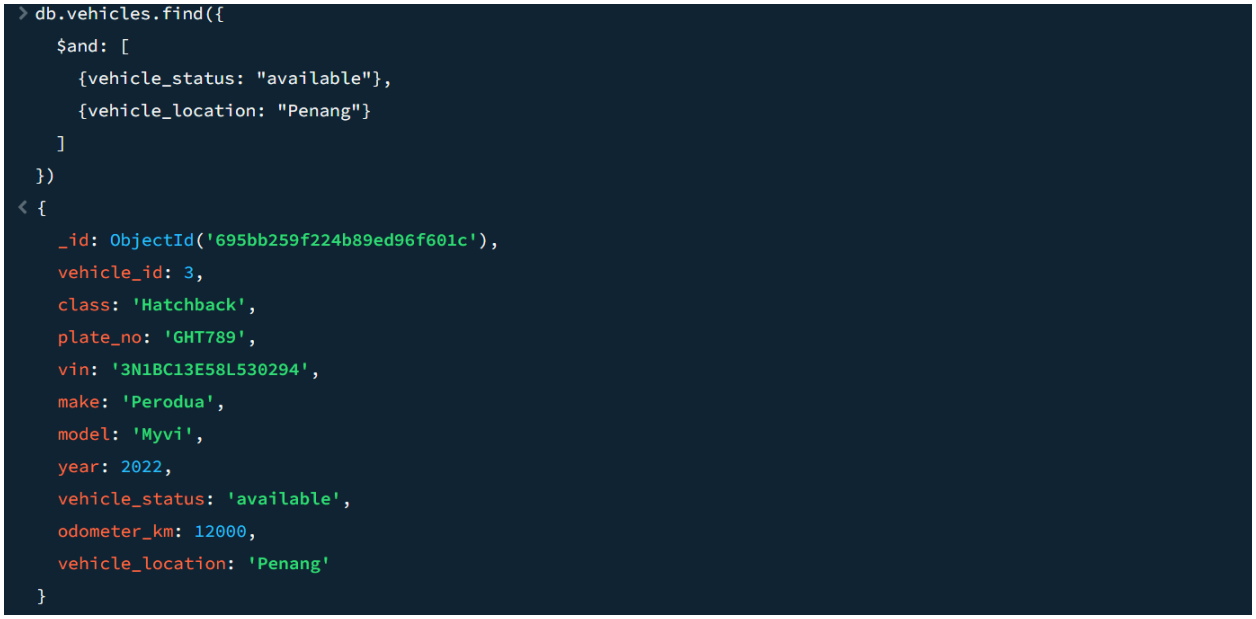


```
    vehicle_status: 'maintenance',
    odometer_km: 15000,
    vehicle_location: 'Johor Bahru'
  }
  {
    _id: ObjectId('695f7335168e10894e69fa76'),
    vehicle_id: 12,
    class: 'MPV',
    plate_no: 'WWB5678',
    vin: 'MPV987654321',
    make: 'Toyota',
    model: 'Veloz',
    year: 2023,
    vehicle_status: 'maintenance',
    odometer_km: 15000,
    vehicle_location: 'Johor Bahru'
  }
```

3.2.3.4 Filtering vehicles that locates in Penang AND available using $and

```JSON
db.vehicles.find({
  $and: [
    {vehicle_status: "available"},
    {vehicle_location: "Penang"}
  ]
})
```

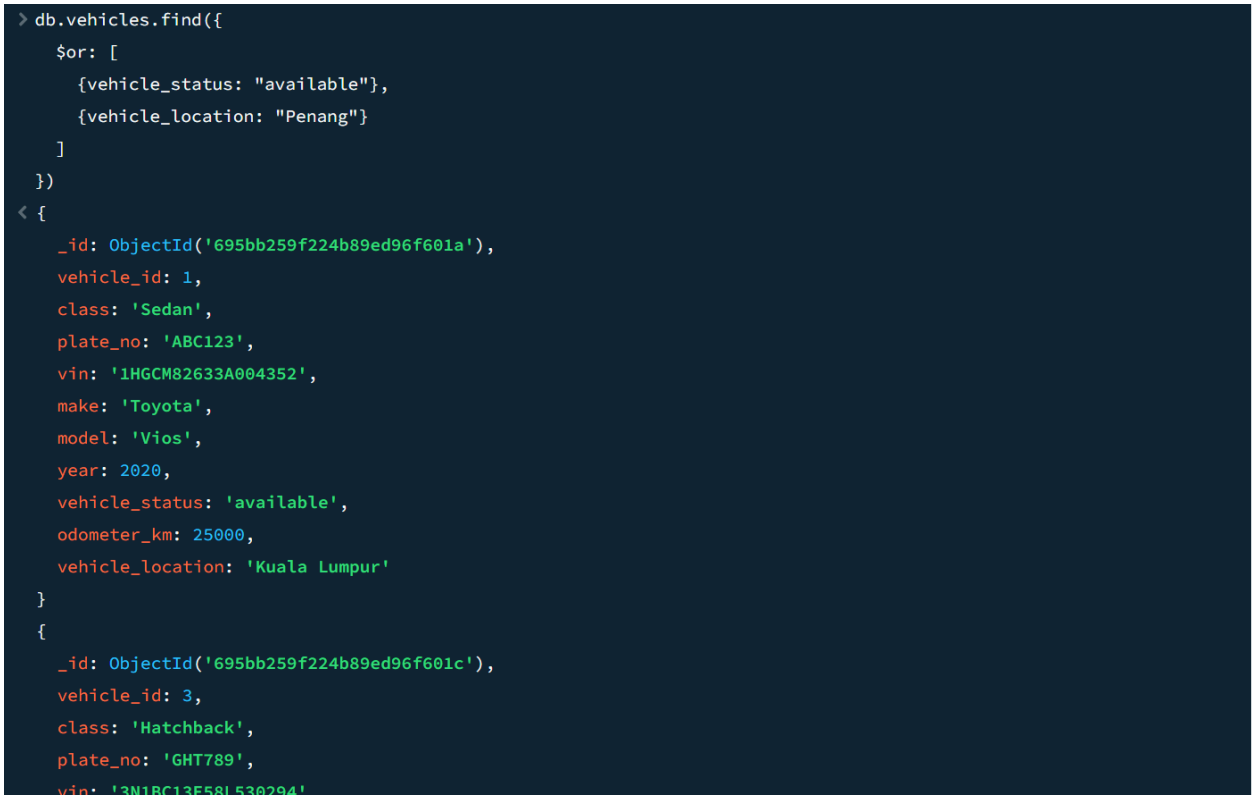
This statement outputs the vehicle that satisfies both conditions, that is vehicle located at Penang and available at the same time.



3.2.3.5 Filtering vehicles that locates in Penang OR available using $or

```
JSON
db.vehicles.find({
  $or: [
    {vehicle_status: "available"},
    {vehicle_location: "Penang"}
  ]
})
```

This statement is almost the same as statement 2.3.4. Using $or, the statement outputs all vehicles that satisfies either one or both of the conditions.



3.2.4 Show customer list that contains only name, email and phone number.

```JSON
db.customers.find(
  {},
  {_id: 0, name: 1, email: 1, phone_no: 1}
)
```

0 here means to exclude the field while 1 means to include the field in the result of find(). Hence, only the name, email and phone number of the result is shown.

3.3 Update Operations

3.3.1 Updating vehicle 1's status to maintenance using updateOne() and set()

```
JSON
db.vehicles.updateOne(
  {vehicle_id: 1},
  {$set: { vehicle_status: "maintenance"}}
)
```

```
> db.vehicles.updateOne(
    {vehicle_id: 1},
    {$set: { vehicle_status: "maintenance"}}
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

3.3.2 Updating all TNG payment that are pending into paid using updateMany() and set()

```
JSON
db.payments.updateMany(
  {method: "TNG", payment_status: "pending"},
  {$set: {payment_status: "paid"}}
)
```

```
> db.payments.updateMany(
    {method: "TNG", payment_status: "pending"},
    {$set: {payment_status: "paid"}}
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 2,
    modifiedCount: 2,
    upsertedCount: 0
  }
```

3.3.3 Update Operators

3.3.3.1 Incrementing the odometer by 500km for vehicle ID 2 using $inc

```JSON
db.vehicles.updateOne(
  {vehicle_id: 2},
  {$inc: {odometer_km: 500}}
)
```

```
> db.vehicles.updateOne(
    {vehicle_id: 2},
    {$inc: {odometer_km: 500}}
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

3.3.3.2 Adding a new phone number for customer ID 2 using $push

```JSON
db.customers.updateOne(
  {customer_id: 2},
  {$push: {phone_no: "01122334455"}}
)
```

```
> db.customers.updateOne(
    {customer_id: 2},
    {$push: {phone_no: "01122334455"}}
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

3.3.3.3 Removing a phone number from customer ID 1 using $pull

```
JSON
db.customers.updateOne(
  {customer_id: 1},
  {$pull: {phone_no: "0123456789"}}
)
```

```
> db.customers.updateOne(
    {customer_id: 1},
    {$pull: {phone_no: "0123456789"}}
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

3.4 Delete Operations

3.4.1 Deleting one record from payments by payment ID using deleteOne()

```
JSON
db.payments.deleteOne({payment_id: 10})
```

```
> db.payments.deleteOne({payment_id: 10})
< {
    acknowledged: true,
    deletedCount: 1
  }
```

3.4.2 Deleting many cancelled booking records at once using deleteMany()

```
JSON
db.bookings.deleteMany({booking_status: "cancelled"})
```

```
> db.bookings.deleteMany({booking_status: "cancelled"})
< {
    acknowledged: true,
    deletedCount: 3
  }
```

4.0 Advanced Operations

4.1 Indexing Implementation

4.1.1 Single-Field Index

```
JSON
db.vehicles.createIndex({vehicle_status: 1})
```

```
> db.vehicles.createIndex({vehicle_status: 1})
< vehicle_status_1
```

This query creates a single-field index on the *vehicle_status* field in the vehicles collection. The value 1 indicates an ascending order index. The vehicle_status field is commonly used to filter vehicles based on their availability, such as available, rented or maintenance. By indexing this field, MongoDB can quickly retrieve vehicles with a specific status, improving query performance during availability checks.

4.1.2 Compound Index

```
JSON
db.vehicles.createIndex({vehicle_location: 1, vehicle_status: 1})
```

```
> db.vehicles.createIndex({vehicle_location: 1, vehicle_status: 1})
< vehicle_location_1_vehicle_status_1
```

This query creates a compound index on the *vehicle_location* and *vehicle_status* fields. This compound index is useful for queries that search for vehicles based on both location and availability status. For example, users often search for available vehicles in a specific city. Indexing these fields together allows MonoDB to narrow down results more efficiently and reduce query execution time.

4.1.3 Query Execution Using Indexed Fields

```
> db.vehicles.find({
    vehicle_location: "Kuala Lumpur",
    vehicle_status: "available"
  })
< {
    _id: ObjectId('695bb259f224b89ed96f601a'),
    vehicle_id: 1,
    class: 'Sedan',
    plate_no: 'ABC123',
    vin: '1HGCM82633A004352',
    make: 'Toyota',
    model: 'Vios',
    year: 2020,
    vehicle_status: 'available',
    odometer_km: 25000,
    vehicle_location: 'Kuala Lumpur'
  }
```

Instead of performing a full collection scan, MongoDB can use the index to quickly locate matching documents, resulting in faster query execution.

4.1.4 Index Verification

```
JSON
db.vehicles.getIndexes()
```

```
> db.vehicles.getIndexes()
< [
    { v: 2, key: { _id: 1 }, name: '_id_' },
    { v: 2, key: { vehicle_status: 1 }, name: 'vehicle_status_1' },
    {
      v: 2,
      key: { vehicle_location: 1, vehicle_status: 1 },
      name: 'vehicle_location_1_vehicle_status_1'
    }
  ]
```

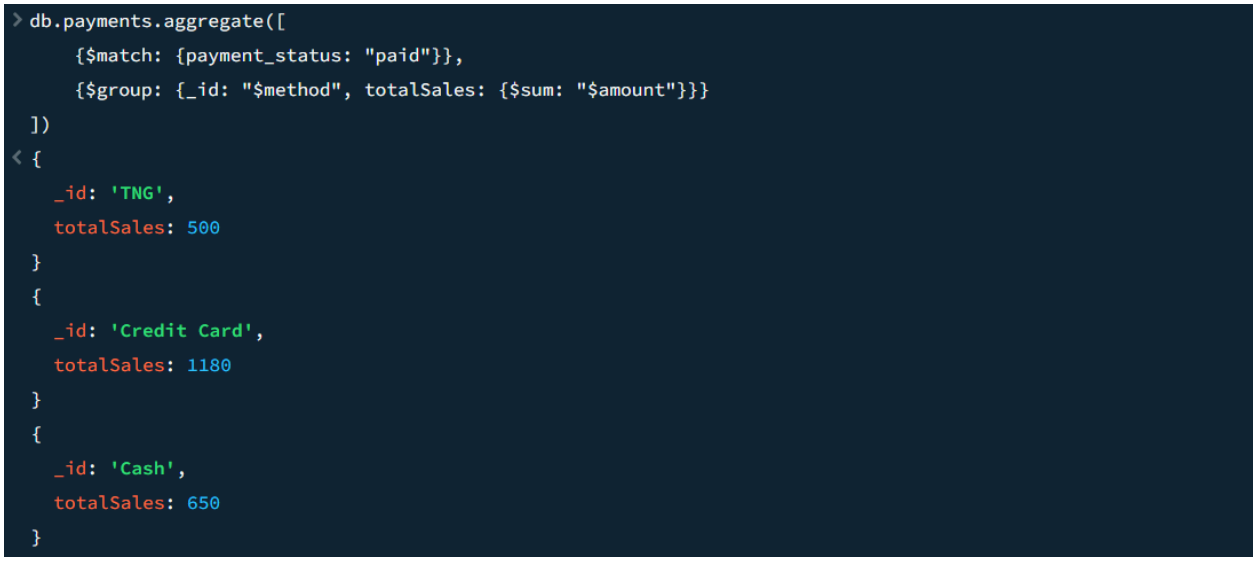
The output confirms that both the single-field index and compound index were successfully created.

4.2 Aggregation Pipelines

4.2.1 Total revenue for “paid” bookings grouped by payment method

```
JSON
db.payments.aggregate([
    {$match: {payment_status: "paid"}},
    {$group: {_id: "$method", totalSales: {$sum: "$amount"}}}
])
```

This query filters for “paid” payments and sums the “amount” for each “method”, showing total revenue by payment method. The following shows the result.



4.2.2 Total payment amount within a time range, grouped by payment status

```
JSON
db.payments.aggregate([
    {$match: {payment_date: {$gte: ISODate("2025-04-01T00:00:00Z"), $lt:
ISODate("2025-05-01T00:00:00Z")}}},
      {$group: {_id: "$payment_status", totalSales: {$sum: "$amount"}}}
])
```

This query filters the “payments” collection to include only payments within a specified month. For April, it would be greater than or equal to the firstmost second of the month and less than the firstmost second of the next month. Then, it is grouped by “payment_status” and sums the “amount” for each status. The following shows the result.

```
> db.payments.aggregate([
    {$match: {payment_date: {$gte: ISODate("2025-04-01T00:00:00Z"), $lt: ISODate("2025-05-01T00:00:00Z")}}},
    {$group: {_id: "$payment_status", totalSales: {$sum: "$amount"}}}
])
< {
  _id: 'refunded',
  totalSales: 260
}
{
  _id: 'paid',
  totalSales: 1100
}
{
  _id: 'pending',
  totalSales: 320
}
```

4.2.3 Bookings with non paid payment statuses

JSON

```
db.payments.aggregate([
    {$match: {payment_status: {$ne: "paid"}}},
    {$lookup: {
        from: "bookings",
        localField: "booking_id",
        foreignField: "booking_id",
        as: "booking"
    }},
    {$unwind: "$booking"},
    {$project: {
        _id: 0,
        booking_id: 1,
        payment_id: 1,
        payment_status: 1,
        method: 1,
        amount: 1,
        payment_date: 1,
        booking_status: "$booking.booking_status",
        pickup_date: "$booking.pickup.date",
        return_date: "$booking.return.date"
    }}
])
```

This query filters the “payments” collection for payments that are not “paid”. Then, it is joined with the “bookings” collection using “booking_id” as the key. Finally, some relevant fields are selected from both collections. The following is a snippet of the result.


```
{
  payment_id: 3,
  booking_id: 3,
  amount: 500,
  method: 'TNG',
  payment_date: 2025-01-04T00:00:00.000Z,
  payment_status: 'pending',
  booking_status: 'cancelled',
  pickup_date: 2025-02-01T00:00:00.000Z,
  return_date: 2025-02-05T00:00:00.000Z
}
{
  payment_id: 6,
  booking_id: 6,
  amount: 350,
  method: 'TNG',
  payment_date: 2025-03-15T00:00:00.000Z,
  payment_status: 'pending',
  booking_status: 'cancelled',
  pickup_date: 2025-03-20T00:00:00.000Z,
  return_date: 2025-03-22T00:00:00.000Z
}
{
  payment_id: 8,
  booking_id: 8,
  amount: 260,
  method: 'Cash',
  payment_date: 2025-04-05T00:00:00.000Z,
  payment_status: 'refunded',
  booking_status: 'cancelled',
  pickup_date: 2025-04-05T00:00:00.000Z,
```

4.2.4 Deposit refund status count and total deposit amount

```
JSON
db.payments.aggregate({
      $group: {_id: "$deposit.refund.status", totalAmount: {$sum:
"$deposit.amount"}, count: {$sum: 1}}
})
```

This query groups the payments collection by “deposit.refund.status” and calcutta the total deposit amount and the number of payments for each status. The following shows the result.

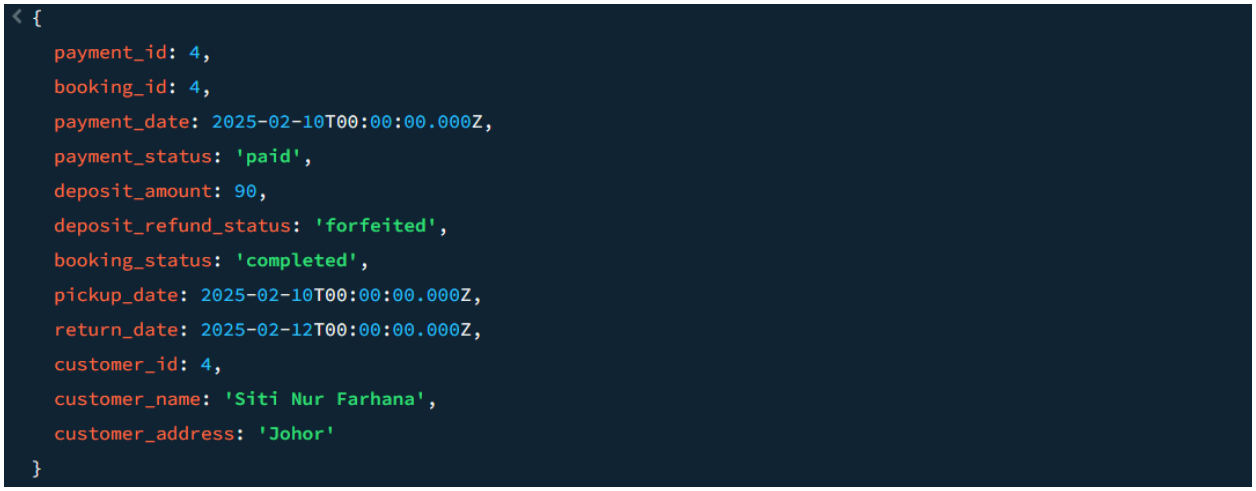
```
> db.payments.aggregate({
    $group: {_id: "$deposit.refund.status", totalAmount: {$sum: "$deposit.amount"}, count: {$sum: 1}}
})
< {
  _id: 'forfeited',
  totalAmount: 90,
  count: 1
}
{
  _id: 'refunded',
  totalAmount: 670,
  count: 5
}
{
  _id: 'processing',
  totalAmount: 355,
  count: 3
}
{
  _id: null,
  totalAmount: 85,
  count: 1
}
```

4.2.5 Customers with forfeited deposits

```
JSON
db.payments.aggregate([
    {$match: {"deposit.refund.status": "forfeited"}},
    {$lookup: {
        from: "bookings",
        localField: "booking_id",
        foreignField: "booking_id",
        as: "booking"
    }},
    {$unwind: "$booking"},
    {$lookup: {
        from: "customers",
        localField: "booking.customer_id",
        foreignField: "customer_id",
        as: "customer"
    }},
    {$unwind: "$customer"},
    {$project: {
        _id: 0,
        payment_id: 1,
        booking_id: 1,
        payment_date: 1,
        payment_status: 1,
        deposit_amount: "$deposit.amount",
        deposit_refund_status: "$deposit.refund.status",
        booking_status: "$booking.booking_status",
        pickup_date: "$booking.pickup.date",
        return_date: "$booking.return.date",
        customer_id: "$customer.customer_id",
        customer_name: "$customer.name",
        customer_address: "$customer.address"
    }}
])
```

This query identifies customers who have forfeited their deposits.
From the “payments” collection, we can see the “booking_id” and from the “bookings” collection, we can see the “customer_id”. Therefore, these 3 collections are joined.
Firstly, we filter the documents with deposit refund status “forfeited”, then we join the filtered payments with collection “bookings” using “booking_id”. After that, we join the results, with the

"customers" collection using "customer_id". Finally, we project the fields that are significant. The following is the result.



4.2.6 Most rented vehicle class

```JSON
db.bookings.aggregate([
    {$match: {booking_status: "completed"}},
    {$lookup: {
        from: "vehicles",
        localField: "vehicle_id",
        foreignField: "vehicle_id",
        as: "vehicle"
    }},
    {$unwind: "$vehicle"},
    {$group: {_id: "$vehicle.class", count: {$sum: 1}}},
    {$sort: {count: -1}}
])
```

This query identifies the most rented vehicle classes, which can help the company understand vehicle popularity and guide future investments.

Firstly, we only select the bookings that are "completed". "cancelled" and "booked" bookings are excluded because they do not reflect actual usage. The filtered bookings are joined with the "vehicles" collection using the "vehicle_id". Then, we count the bookings by its vehicle class and sort the results in descending order to show the most rented classes first. The following shows the result.

```
> db.bookings.aggregate([
    {$match: {booking_status: "completed"}},
    {$lookup: {
        from: "vehicles",
        localField: "vehicle_id",
        foreignField: "vehicle_id",
        as: "vehicle"
    }},
    {$unwind: "$vehicle"},
    {$group: {_id: "$vehicle.class", count: {$sum: 1}}},
    {$sort: {count: -1}}
])
< {
  _id: 'Sedan',
  count: 3
}
{
  _id: 'SUV',
  count: 2
}
{
  _id: 'Van',
  count: 1
}
```

4.2.7 The top 3 vehicle make owned

```JSON
db.vehicles.aggregate([
    {$group: {_id: "$make", count: {$sum: 1}}},
    {$sort: {count: -1}},
    {$limit: 3}
])
```

This query gives an overview of the most common vehicle makes in the company. From the “vehicles” collection, we group the documents with their make and count the number of vehicles belonging to each make. Then we sort the counts in descending order to see the most common makes and limit the output to the top 3 makes.

```
> db.vehicles.aggregate([
    {$group: {_id: "$make", count: {$sum: 1}}},
    {$sort: {count: -1}},
    {$limit: 3}
])
< {
  _id: 'Honda',
  count: 2
}
{
  _id: 'Toyota',
  count: 2
}
{
  _id: 'Perodua',
  count: 2
}
```

4.3 Sorting Demonstration

4.3.1 Sorting Customers' Contact Information by Name (Ascending)

```
JSON
db.customers.find({},{_id:0,name:1,email:1,phone_no:1}).sort({name:1})
```

This query displays the customer names along with their contact details (email and phone number) in ascending order by name. Only the name, email, and phone fields are shown.

```
> db.customers.find({},{_id:0,name:1,email:1,phone_no:1}).sort({name:1})
< {
  name: 'Chong Kai Wen',
  email: 'kaiwen.chong@example.com',
  phone_no: [
    '0190123456'
  ]
}
{
  name: 'Harish A/L Rajendran',
  email: 'harish.raj@example.com',
  phone_no: [
    '0112345678',
    '0165566778'
  ]
}
{
  name: 'Jamie A/L Arjun Kumar',
  email: 'arjun.kumar@example.com',
  phone_no: [
    '0145678901',
    '0193344556'
  ]
}
```

4.3.2 Sorting Paid Payment by Amount (Descending)

```
JSON
db.payments.find
  ({payment_status: "paid"},
   {_id:0,payment_id:1,amount:1,method:1,payment_date:1,payment_status:1})
  .sort({amount:-1})
```

This query filters the payment in “payments collection” with “paid” status only and displays payment id, amount, payment method, payment date and payment status, sorted by payment amount in descending order.

```
> db.payments.find
    ({payment_status: "paid"},
     {_id:0,payment_id:1,amount:1,method:1,payment_date:1,payment_status:1})
    .sort({amount:-1})
< {
    payment_id: 7,
    amount: 600,
    method: 'Credit Card',
    payment_date: 2025-04-01T00:00:00.000Z,
    payment_status: 'paid'
  }
  {
    payment_id: 9,
    amount: 500,
    method: 'TNG',
    payment_date: 2025-04-12T00:00:00.000Z,
    payment_status: 'paid'
  }
  {
    payment_id: 5,
    amount: 400,
    method: 'Cash',
    payment_date: 2025-03-15T00:00:00.000Z,
    payment_status: 'paid'
  }
```

4.4 Explanation of Performance Improvements

The combination of indexing, aggregation pipelines, and sorting operations are used in order to improve performance in the car rental system. Indexing enhances the speed in which queries are served by enabling MongoDB to search and find the appropriate documents promptly without necessarily retrieving the entire collections. Indexes formed on the often requested fields like the vehicle status and location save a lot of time in execution queries.

Aggregation pipelines are used to improve on performance by filtering and grouping the data directly in the database, which is the purpose of processing and summarizing data. This saves unwarranted data search and assists in effective analytical searches. Sorting also enhances

usability through arrangement of query result in an orderly manner that is meaningful after filtering to enable efficient and structured data access.

Altogether, these enhanced functions improve the performance of queries, scalability, and accessibility of data leading to a more responsive and optimized NoSQL backend.

5.0 Security & Limitations

5.1 Basic Access Control

In the car rental system, security is mainly focused on protecting sensitive data stored in the Customers and Payments collections, such as personal and financial data. Therefore, MongoDB's role-based access control can be used to limit read and write access in these collections. Less sensitive collections like Vehicles and Bookings require fewer restrictions.

5.2 Injection Risk

Although MongoDB is not vulnerable to traditional SQL injection attacks, NoSQL injection still may occur if user inputs are directly included in query conditions. In our project, all CRUD and advanced database operations are executed using predefined queries in MongoDB compass. This can help to reduce the risk of injection attacks by preventing malicious query manipulation.

5.3 Database Limitations

While MongoDB provides flexibility with a dynamic schema data model, it may result in inconsistent document structure when validation rules are not enforced. The data redundancy can happen if the same information is stored in multiple collections, which increases the risk of inconsistency during updates. Besides, MongoDB does not support native joins like relational databases and cross-collection queries that require aggregation pipelines can lead to increased query complexity.

In distributed systems, MongoDB follows an eventual consistency model; if no new updates are made to the data, all replicas will eventually converge to the same value. This may cause temporary inconsistencies between related collections such as Bookings and Payments, which represents a trade-off between data consistency, scalability, and performance.

6.0 Teamwork

Team Member	Contributions and Reflections
Teh Ru Qian	I was responsible for the introduction and designing of the NoSQL data model. I explained the project domain, objectives and created the document structure using embedding, referencing and arrays providing a foundation for the rest of the team's work. I learned how to represent real-world data in MongoDB using embedded documents, arrays and references. I also practiced keeping data accurate, retrieving it quickly, handling relationships and organizing documents.
Tan Yi Ya	I was responsible for constructing and testing the CRUD operations for the System. I wrote scripts for data insertion, complex filtering using operators like $regex and $lte, and managed specific field visibility using projections to ensure efficient data handling. I learned how to query nested documents using dot notation and the strict rules of inclusion and exclusion in projections. I also gained knowledge for interpreting update results and understanding the difference between matching a document and actually modifying its data.
Woo Cheng Shuan	I was responsible for creating indexes to improve data retrieval efficiency for users. Through this workload, I learned how indexing can significantly reduce query execution time. After completing the advanced operations, I conducted a brief discussion on query performance. By observing and understanding the queries implemented by my teammates, I gained better insight how optimized indexing supports faster and more efficient data access in MongoDB, and how aggregation pipelines and sorting operations can further enhance data analysis and result organization. By learning from the queries implemented by my teammates, I gained a deeper understanding of how different advanced operations work together to improve overall database performance and usability.
Lam Yoke Yu	I was responsible for the aggregation pipelines. I created seven queries that use different combinations of pipeline stages. I also applied $lookup to join documents from multiple collections. Through this process, I get to develop a deeper understanding of aggregation and form a clearer conceptual view of the pipeline works in my mind. In addition, I learned to use $lookup. Although I initially relied on AI tools for guidance, I am now able to write these queries

	independently without external assistance.
Goe Jie Ying	I was responsible for implementing data sorting operations in MongoDB, demonstrating both ascending and descending order queries on collections. In addition, I was also responsible for discussing security measures and database limitations. Through these tasks, I was able to know how sorting helps to organize data for better analysis and reporting. I also learned about the security issues and limitations in a NoSQL database. By working alongside my teammates, I developed a deeper understanding of how different basic and advanced operations work together to improve overall database efficiency and how they make data analysis and reporting easier.

7.0 Conclusion

In this project, our team successfully deployed the car rental system to a NoSQL document-oriented model of MongoDB. Through this work, we gained hands-on experience in applying document data modeling techniques such as embedding and referencing. This strengthened our understanding of how MongoDB's flexible schema can adapt more easily to changing business requirements compared to traditional SQL-based systems.

One of the primary challenges encountered was performing complex data analysis using MongoDB's Aggregation Pipeline, especially when using $lookup and $unwind to connect disparate data across multiple collections. We also had to carefully balance the trade-offs between faster read performance through embedding and the risk of data redundancy and outdated information.

To optimize the system, we applied strategic indexing on frequently queried fields like vehicle_location and vehicle_status. This significantly reduced query execution time by avoiding full collection scans. This experience highlighted the importance of designing database architecture based on real-world user behavior, ensuring that the backend remains responsive as the volume of rental transactions grows.

For future improvements, it is recommended to implement MongoDB Schema Validation to prevent inconsistent or invalid data stored. Additionally, using Change Streams could help maintain real-time consistency between the Bookings and Payments collections, ensuring that status updates are reflected across the system instantly. Overall, this project provided valuable exposure to modern NoSQL database design and optimization and strengthened our readiness to develop scalable and high-performance backend systems.

Video Link: https://youtu.be/DfEJ5hfovSI

Code (zip file is also attached upon submission)**:**
https://github.com/tanyiya/DP-Project/tree/f87fa7f233b54a49facbd7add301122529020cf3/Project%20II